



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

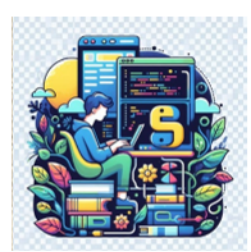
PROGRAMMING WITH PYTHON

Object-Oriented Programming (OOP) and Advanced Concepts

Methods and Attributes

Lekha A

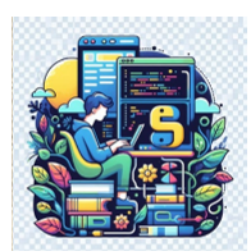
Computer Applications



PROGRAMMING WITH PYTHON

Recap

- Classes
- Objects



PROGRAMMING WITH PYTHON

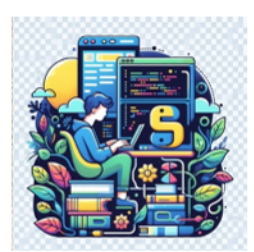
Accessing Attributes

- Example

```
print("I own a " + laptop1.brand + " " + laptop1.model + " that costs ₹" + str(laptop1.price) + ".")  
print("I own a " + laptop2.model + " from " + laptop2.brand + ".")
```

1

```
I own a Dell Inspiron 15 that costs ₹55000.  
I own a MacBook Air from Apple.
```

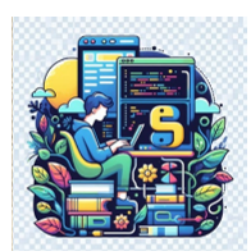


PROGRAMMING WITH PYTHON

Accessing Attributes

- To access the attributes of an instance, dot notation is used.
- At **1** access the value of laptop1 attribute brand, model, price by writing

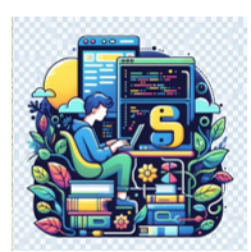
```
print("I own a " + laptop1.brand + " " + laptop1.model + " that costs ₹" +  
str(laptop1.price) + ".")
```
- Dot notation is used often in Python.



PROGRAMMING WITH PYTHON

Accessing Attributes

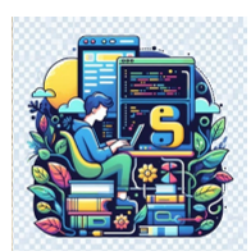
- This syntax demonstrates how Python finds an attribute's value.
- Here Python looks at the instance `laptop1` and then finds the attribute associated with `laptop1`.



PROGRAMMING WITH PYTHON

Accessing Attributes

- This is the same attribute referred to as `self.brand` in the class `Laptop`.
- In the print statement, `str(laptop1.price)` converts 55000, the value of `laptop1`'s price attribute to a string.



PROGRAMMING WITH PYTHON

Calling Methods

- Examples

```
laptop1.turn_on()  
laptop1.display_info()  
laptop1.turn_off()
```

Dell Inspiron 15 is now ON.

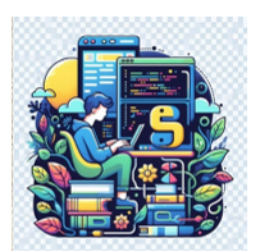
Laptop Details:

Brand: Dell

Model: Inspiron 15

Price: ₹55000

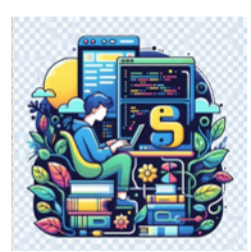
Dell Inspiron 15 is now OFF.



PROGRAMMING WITH PYTHON

Calling Methods

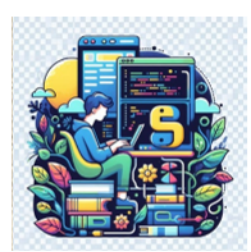
- After an instance from the class Laptop is created, dot notation can be used to call any method defined in Laptop.
- To call a method, give the name of the instance (in this case, laptop1) and the method that needs to be called, separated by a dot.



PROGRAMMING WITH PYTHON

Calling Methods

- When Python reads `laptop1.turn_on()` it looks for the method `turn_on()` in the class `Laptop` and runs that code.
- Python interprets the line `laptop1.turn_off()` in the same way.



PROGRAMMING WITH PYTHON

Creating Multiple Instances

- Each laptop is a separate instance with its own set of attributes, capable of the same set of actions

```
laptop2.turn_on()  
laptop2.display_info()  
laptop2.turn_off()
```

Apple MacBook Air is now ON.

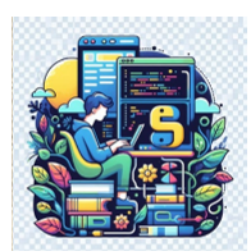
Laptop Details:

Brand: Apple

Model: MacBook Air

Price: ₹120000

Apple MacBook Air is now OFF.



PROGRAMMING WITH PYTHON

Setting a Default Value for an Attribute

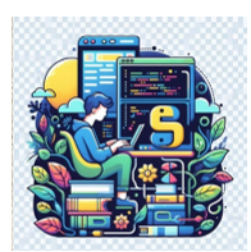
- Add another method

```
class Laptop:
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
        self.price = price

    def turn_on(self):
        print(f"{self.brand} {self.model} is now ON.")

    def turn_off(self):
        print(f"{self.brand} {self.model} is now OFF.")

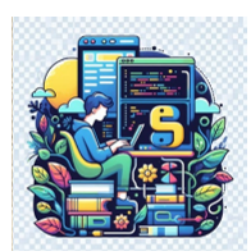
    def display_info(self):
        print(f"Laptop Details:\nBrand: {self.brand}\nModel: {self.model}\nPrice: ₹{self.price}")
```



PROGRAMMING WITH PYTHON

Setting a Default Value for an Attribute

- Every attribute in a class needs an initial value, even if that value is 0 or an empty string.
- Add an attribute called `battery_level` that always starts with a value of 100.



PROGRAMMING WITH PYTHON

Setting a Default Value for an Attribute

```
class Laptop:
```

```
    def __init__(self, brand, model, price):
```

```
        self.brand = brand
```

```
        self.model = model
```

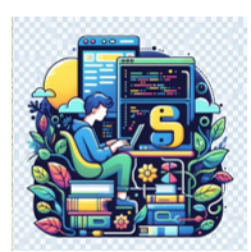
```
        self.price = price
```

```
        self.battery_level = 100
```

```
    def check_battery(self):
```

```
        """Print a statement showing the current battery level."""
```

```
        print(f"This laptop has {self.battery_level}% battery remaining.")
```

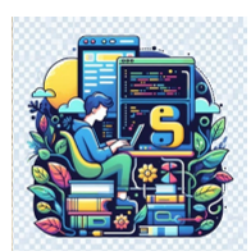


PROGRAMMING WITH PYTHON

Setting a Default Value for an Attribute

- `laptop1.check_battery()`

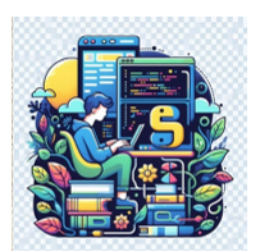
This laptop has 100% battery remaining.



PROGRAMMING WITH PYTHON

Modifying Attribute Values

- An attribute's value can be changed in three ways
 - Change the value directly through an instance
 - Set the value through a method
 - Increment the value (add a certain amount to it) through a method



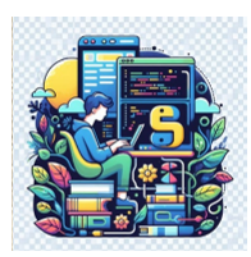
PROGRAMMING WITH PYTHON

Modifying an attribute value directly

- The simplest way to modify the value of an attribute is to access the attribute directly through an instance.

```
laptop2.turn_on()  
print(laptop2.get_descriptive_info())  
laptop2.battery_level = 85  
laptop2.check_battery()  
laptop2.turn_off()
```

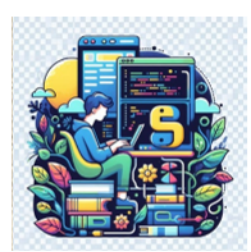
Apple MacBook Air is now turned ON.
Apple Macbook Air Priced At ₹120000
This laptop has 85% battery remaining.
Apple MacBook Air has been shut down.



PROGRAMMING WITH PYTHON

Modifying an attribute value using a method

- Have methods that update certain attributes for the user.
- Instead of accessing the attribute directly, pass the new value to a method that handles the updating internally.



PROGRAMMING WITH PYTHON

Modifying an attribute value using a method

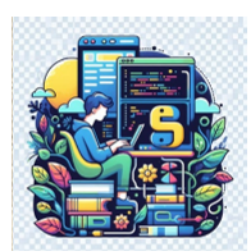
```
def update_battery_level(self, new_level):  
    """Set the battery level to the given value."""  
    if 0 <= new_level <= 100:  
        self.battery_level = new_level  
        print(f"Battery level updated to {self.battery_level}%.")  
    else:  
        print("Invalid battery level! Please enter a value between 0 and 100.")
```

```
laptop1.update_battery_level(75)  
laptop1.check_battery()
```

This laptop has 75% battery remaining.

```
laptop1.update_battery_level(120)
```

Invalid battery level! Please enter a value between 0 and 100.



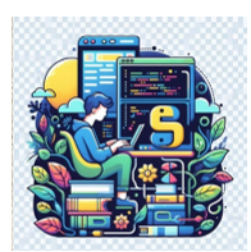
PROGRAMMING WITH PYTHON

Incrementing an Attribute's Value Through a Method

```
def decrease_battery(self, usage):  
    """Reduce the battery level by the given amount."""  
    if usage < 0:  
        print("Usage amount cannot be negative!")  
    elif self.battery_level - usage < 0:  
        print("Battery too low! Please charge your laptop.")  
        self.battery_level = 0  
    else:  
        self.battery_level -= usage  
        print(f"Battery decreased by {usage}%. Current level: {self.battery_level}%")
```

```
laptop1.decrease_battery(15)  
laptop1.decrease_battery(50)  
laptop1.check_battery()
```

Battery decreased by 15%. Current level: 85%.
Battery decreased by 50%. Current level: 35%.



PROGRAMMING WITH PYTHON

Recap

- Accessing Attributes
- Calling Methods
- Creating Multiple Instances
- Setting a Default Value for an Attribute
- Modifying Attribute Values



THANK YOU

Lekha A
Computer Applications